Task 1: Determine if String Halves Are Alike – Java Program

Aim

To write a Java program to determine whether the **two halves of a string contain the same number of vowels**.

Algorithm

1. Start the program.
2. Read a string from the user.
3. Find the middle position of the string.
4. Divide the string into two equal halves.
5. Count the vowels in the first half.
6. Count the vowels in the second half.
7. Compare the two vowel counts.
8. If both counts are equal, print **"Halves are Alike"**.
9. Otherwise, print **"Halves are Not Alike"**.
10. Stop the program.

Program

```
import java.util.Scanner;


public class StringHalvesAlike {


   static int countVowels(String str) {
      int count = 0;


      for (int i = 0; i < str.length(); i++) {
         char ch = Character.toLowerCase(str.charAt(i));


         if (ch == 'a' || ch == 'e' || ch == 'i' ||
            ch == 'o' || ch == 'u') {
```

```
            count++;
        }
    }

    return count;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter a string: ");
    String str = sc.nextLine();

    int mid = str.length() / 2;

    String firstHalf = str.substring(0, mid);
    String secondHalf = str.substring(mid);

    int firstCount = countVowels(firstHalf);
    int secondCount = countVowels(secondHalf);

    if (firstCount == secondCount) {
        System.out.println("Halves are Alike");
    } else {
        System.out.println("Halves are Not Alike");
    }

    sc.close();
```

```
    }
}
```

Output

Example 1:

Enter a string: book

Halves are Alike

Example 2:

Enter a string: textbook

Halves are Not Alike

Result

Thus, the Java program to **determine whether the two halves of a string are alike based on the number of vowels** was successfully executed.

Task 2: Contains Duplicate – Java Program

Aim

To write a Java program to determine whether an array contains **any duplicate elements**.

Algorithm

1. Start the program.
2. Read the number of elements in the array.
3. Read the array elements.
4. Create a HashSet to store unique elements.
5. Traverse through each element of the array.
6. If an element already exists in the set, a duplicate is found.
7. Otherwise, add the element to the set.
8. Display true if a duplicate exists; otherwise display false.
9. Stop the program.

Program

```
import java.util.HashSet;
import java.util.Scanner;


public class ContainsDuplicate {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      System.out.print("Enter number of elements: ");
      int n = sc.nextInt();


      int[] nums = new int[n];


      System.out.println("Enter the elements:");
```

```
        for (int i = 0; i < n; i++) {
            nums[i] = sc.nextInt();
        }

        HashSet<Integer> set = new HashSet<>();
        boolean duplicate = false;

        for (int num : nums) {
            if (set.contains(num)) {
                duplicate = true;
                break;
            }

            set.add(num);
        }

        System.out.println("Contains Duplicate: " + duplicate);

        sc.close();
    }
}
```

Output

Example 1:

Enter number of elements: 5

Enter the elements:

1 2 3 1 4

Contains Duplicate: true

Example 2:

Enter number of elements: 5

Enter the elements:

1 2 3 4 5

Contains Duplicate: false

Result

Thus, the Java program to **determine whether an array contains duplicate elements** was successfully executed and the correct result was obtained.

Task 3: Move Zeroes – Java Program

Aim

To write a Java program to **move all zeroes in an array to the end while maintaining the relative order of non-zero elements**.

Algorithm

1. Start the program.
2. Read the number of elements in the array.
3. Read the array elements.
4. Initialize an index j = 0.
5. Traverse through the array.
6. If the current element is non-zero, place it at index j and increment j.
7. After processing all non-zero elements, fill the remaining positions with zeroes.
8. Display the modified array.
9. Stop the program.

Program

```
import java.util.Scanner;


public class MoveZeroes {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      System.out.print("Enter number of elements: ");
      int n = sc.nextInt();


      int[] nums = new int[n];


      System.out.println("Enter the elements:");
```

```
    for (int i = 0; i < n; i++) {
        nums[i] = sc.nextInt();
    }

    int j = 0;

    // Move non-zero elements to the front
    for (int i = 0; i < n; i++) {
        if (nums[i] != 0) {
            nums[j] = nums[i];
            j++;
        }
    }

    // Fill remaining positions with zero
    while (j < n) {
        nums[j] = 0;
        j++;
    }

    System.out.println("Array after moving zeroes:");

    for (int num : nums) {
        System.out.print(num + " ");
    }

    sc.close();
}
```

}

Output

Enter number of elements: 6

Enter the elements:

0 1 0 3 12 0

Array after moving zeroes:

1 3 12 0 0 0

Result

Thus, the Java program to **move all zeroes to the end of the array while maintaining the order of non-zero elements** was successfully executed and the desired output was obtained.

Task 4: Transpose Matrix – Java Program

Aim

To write a Java program to find the **transpose of a matrix**.

Algorithm

1. Start the program.
2. Read the number of rows and columns of the matrix.
3. Read the matrix elements.
4. Interchange the rows and columns of the matrix.
5. Store the transposed elements in a new matrix.
6. Display the original matrix.
7. Display the transpose matrix.
8. Stop the program.

Program

```
import java.util.Scanner;


public class TransposeMatrix {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);


        System.out.print("Enter number of rows: ");
        int rows = sc.nextInt();


        System.out.print("Enter number of columns: ");
        int columns = sc.nextInt();


        int[][] matrix = new int[rows][columns];
        int[][] transpose = new int[columns][rows];
```

```
        System.out.println("Enter matrix elements:");

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < columns; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        // Find transpose
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < columns; j++) {
                transpose[j][i] = matrix[i][j];
            }
        }

        System.out.println("Transpose of the Matrix:");

        for (int i = 0; i < columns; i++) {
            for (int j = 0; j < rows; j++) {
                System.out.print(transpose[i][j] + " ");
            }
            System.out.println();
        }

        sc.close();
    }
}
```

Output

Enter number of rows: 2

Enter number of columns: 3

Enter matrix elements:

1 2 3

4 5 6

Transpose of the Matrix:

1 4

2 5

3 6

Result

Thus, the Java program to **find the transpose of a given matrix** was successfully executed and the transposed matrix was obtained.

Task 5: Matrix Block Sum – Java Program

Aim

To write a Java program to find the **matrix block sum** for a given matrix and block size k.

Algorithm

1. Start the program.
2. Read the number of rows and columns of the matrix.
3. Read the matrix elements.
4. Read the block size k.
5. For each element, consider the rectangular block from i-k to i+k and j-k to j+k.
6. Make sure the row and column indices remain within the matrix boundaries.
7. Calculate the sum of all elements in the block.
8. Store the sum in the corresponding position of the result matrix.
9. Display the resulting matrix.
10. Stop the program.

Program

```
import java.util.Scanner;


public class MatrixBlockSum {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      System.out.print("Enter number of rows: ");
      int m = sc.nextInt();


      System.out.print("Enter number of columns: ");
      int n = sc.nextInt();


      int[][] mat = new int[m][n];
```

```
int[][] result = new int[m][n];

System.out.println("Enter matrix elements:");

for (int i = 0; i < m; i++) {
   for (int j = 0; j < n; j++) {
      mat[i][j] = sc.nextInt();
   }
}

System.out.print("Enter block size k: ");
int k = sc.nextInt();

for (int i = 0; i < m; i++) {
   for (int j = 0; j < n; j++) {

      int sum = 0;

      int rowStart = Math.max(0, i - k);
      int rowEnd = Math.min(m - 1, i + k);

      int colStart = Math.max(0, j - k);
      int colEnd = Math.min(n - 1, j + k);

      for (int r = rowStart; r <= rowEnd; r++) {
         for (int c = colStart; c <= colEnd; c++) {
            sum += mat[r][c];
         }
```

```
            }

            result[i][j] = sum;
        }
    }

    System.out.println("Matrix Block Sum:");

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            System.out.print(result[i][j] + " ");
        }
        System.out.println();
    }

    sc.close();
  }
}
```

Output

```
Enter number of rows: 3
Enter number of columns: 3
Enter matrix elements:
1 2 3
4 5 6
7 8 9
Enter block size k: 1

Matrix Block Sum:
```

12 21 16

27 45 33

24 39 28

Result

Thus, the Java program to **calculate the matrix block sum for each element using the given block size k** was successfully executed and the required result matrix was obtained.